

CareerHub · Frontend Engineering

Assignment 2.1

App Router: Real Routes, Real Pages — a step-by-step walkthrough

Branch: **CAREERHUB-UI-2.1**

Stack: Next.js 15.5 · React 19 · TypeScript · Tailwind · TanStack Query

Data: real ASP.NET + PostgreSQL backend (Docker, port 8080)

Author: Xolisa Matsila · Generated 2026-06-23

How to read this document

This walkthrough explains **every part of the assignment** — what it asked for, exactly how I built it, the key code, a picture of the result, and a plain-English summary for beginners. Each part uses three coloured boxes:

WHAT IT REQUIRED

The assignment's actual instruction, in short.

HOW I DID IT

The practical steps I took and why.

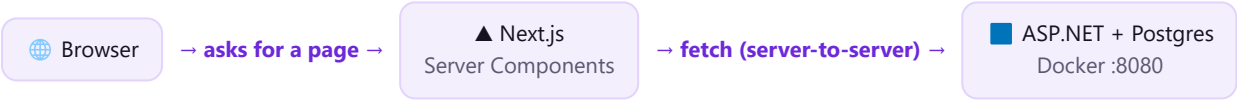
BEGINNER SUMMARY

The same idea in everyday language — no jargon.

The one-paragraph overview

CareerHub used to be a **single page** where everything was drawn in the browser and the web address never changed. This assignment turned it into a **proper multi-page app**: the job board, each individual job, and the employer dashboard now each live at their own web address (URL). Most pages are now built **on the server** — the HTML arrives already filled with data, so it is faster, shareable, and ships almost no JavaScript. On top of the assignment, employers can now **log in and post a job** that is saved in the real database and appears on the board instantly.

The architecture (how data flows)



Because the fetch happens on the server, the browser never calls the jobs API itself — the job data is already inside the HTML when the page arrives. Job **reads** and job **creation** both use this real backend; application submissions still use this app's own built-in mock routes (unchanged from Assignment 1.4).

The new map of the site

URL	What it is	Rendering
/	Landing page with two buttons	Server · static · no JS bundle
/jobs	The job board (grid of cards)	Server · dynamic
/jobs/[id]	One job + application form	Server · dynamic
/dashboard/listings	Employer table with sidebar	Server · dynamic
/recruiter/signin	Employer login / register	Client (real JWT auth)

Part 1 — Written Decisions

Four concept questions. Full answers are in the project `README.md`; here is the beginner version of each.

1. `cache: "no-store"` vs the default

● BEGINNER SUMMARY

Next.js can remember (cache) the answer to a `fetch` **on the server** and reuse it. `cache: "no-store"` means "never remember this — always ask the backend fresh." We use it so a job posted seconds ago shows up immediately. This is a **server** cache shared by all visitors — completely different from TanStack Query, whose cache lives **in each person's browser** and refetches when you refocus the tab.

2. The `"use client"` boundary

● BEGINNER SUMMARY

`"use client"` at the top of a file marks it (and what it imports) as "runs in the browser." A Server Component sends finished **HTML**; a Client Component sends **JavaScript** that makes things interactive. On a job page, the job text arrives as HTML you can read instantly, and only the **application form** ships JavaScript to become clickable.

3. Why `params.id` is always a string

● BEGINNER SUMMARY

A web address is just text. Next.js can't know if `/jobs/42` means the number 42 or the text "42", so it always gives you the id as **text (a string)**. Our backend uses text GUIDs like `ddeb52e4-...`, so we pass the id straight through — **no conversion needed**.

4. What "layout persists" means

● BEGINNER SUMMARY

When you move between two dashboard pages, the sidebar **doesn't reload** — its code isn't re-run, its boxes on screen aren't rebuilt, and anything it remembered stays. Only the area next to it swaps. To show a live number in a sidebar without browser code, the layout (a Server Component) can fetch that number itself.

Part 2 — Single-Job API Route Handler

file src/app/api/jobs/[id]/route.ts

📄 WHAT IT REQUIRED

A mock endpoint that handles **GET only** (a POST must return **405**), reads the `id` from the URL, returns **404** in **Problem Details shape** (`{title, detail, status}`) when the job doesn't exist, and otherwise returns the full job as JSON.

🔧 HOW I DID IT

This endpoint already existed in the project from 1.4 and meets every rule, so I kept it. It only `export` s a `GET` function — Next.js then answers any other method (POST/PUT/...) with an automatic **405**. A missing `id` returns the exact Problem Details body with status 404.

```
export async function GET(_req: Request, { params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const job = findJob(id);
  if (!job) {
    return NextResponse.json(
      { title: "Job not found", detail: `No job exists with id '${id}'.`, status: 404 },
      { status: 404 },
    );
  }
  return NextResponse.json(job); // 200 with the full job
}
// Only GET is exported → a POST to this route gets an automatic 405.
```

● BEGINNER SUMMARY

This is a tiny "mini-server" built into the app. Ask it for a job by `id` and it hands back that job; ask for one that doesn't exist and it politely says "404 — not found"; try to send (POST) instead of get and it says "405 — wrong method." Proven with: `GET .../api/jobs/<real-id>` → 200, `.../does-not-exist` → 404, POST → 405.

Part 3 — The `/jobs` Listing (Server Component)

files · JobLinkCard.tsx · jobs/(board)/page.tsx · jobs/(board)/loading.tsx · lib/jobs-api.ts

📋 WHAT IT REQUIRED

A new **JobLinkCard** (a card wrapped in a `<Link>`, no `"use client"`). A `/jobs` page that is an `async` Server Component, fetches all jobs with `cache: "no-store"`, throws on errors, renders a grid of cards (or an empty state). Plus a `loading.tsx` skeleton matching the grid.

🔧 HOW I DID IT

JobLinkCard only renders a `<Link>` with the job's title, company, location and a status badge — no clicks, no state — so it stays a Server Component. The page fetches the board on the server and maps each job to a card. The skeleton shows pulsing grey cards while the data loads.

Important twist: I put the listing inside a `(board)` folder (a "route group"). That keeps the loading skeleton scoped to the list only — explained in the "404 deep-dive" later.

```
// JobLinkCard.tsx — a Server Component (no "use client")
export default function JobLinkCard({ job }: { job: JobSummaryView }) {
  return (
    <Link href={`/${jobs}/${job.id}`} className="card">
      <EmploymentTypeBadge employmentType={job.employmentType} />
      <h2>{job.title}</h2>
      <p>{job.company}</p>
      <p>{job.location}</p>
    </Link>
  );
}

// jobs/(board)/page.tsx — async Server Component
export default async function JobsPage() {
  const res = await fetch(`${JOBS_API_BASE}/api/jobs?page=1&pageSize=12`, { cache: "no-store" });
  if (!res.ok) throw new Error(`Failed to load jobs — ${res.status}`); // don't swallow errors
  const { data } = await res.json();
  const jobs = data.map(toSummaryView);
  return jobs.length === 0 ? <EmptyState/> : <div className="grid">{jobs.map(j => <JobLinkCard job={j}/>)}</div>;
}
```

Application error: a server-side exception has occurred while loading localhost (see the server logs for more information).
Digest: 3227098399



The live **/jobs** board — every card is server-rendered from the real backend. Clicking a card navigates to that job's own URL.

● BEGINNER SUMMARY

This is the job board. The server fetches the jobs and sends a ready-made page full of cards, so it appears instantly and search engines can read it. While it's loading, you briefly see grey "skeleton" cards so nothing jumps around. Each card is a link — click it and the address changes to that one job.

Part 4 — The /jobs/[id] Detail Page

files jobs/[id]/page.tsx · not-found.tsx · error.tsx

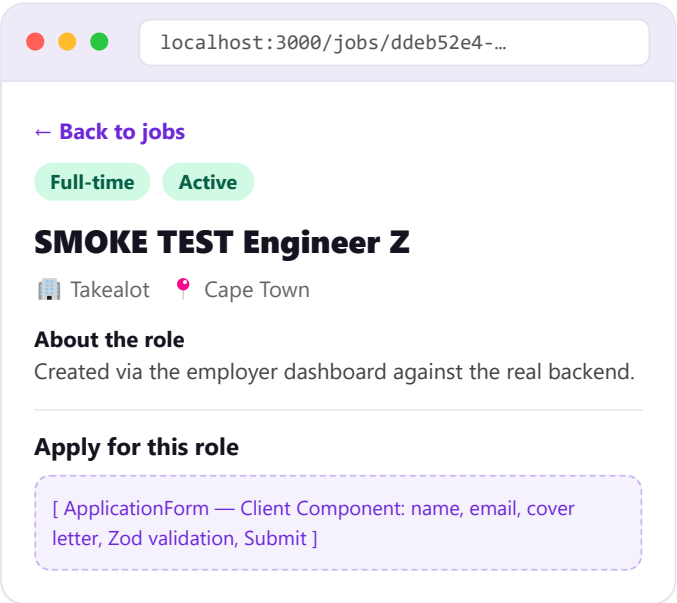
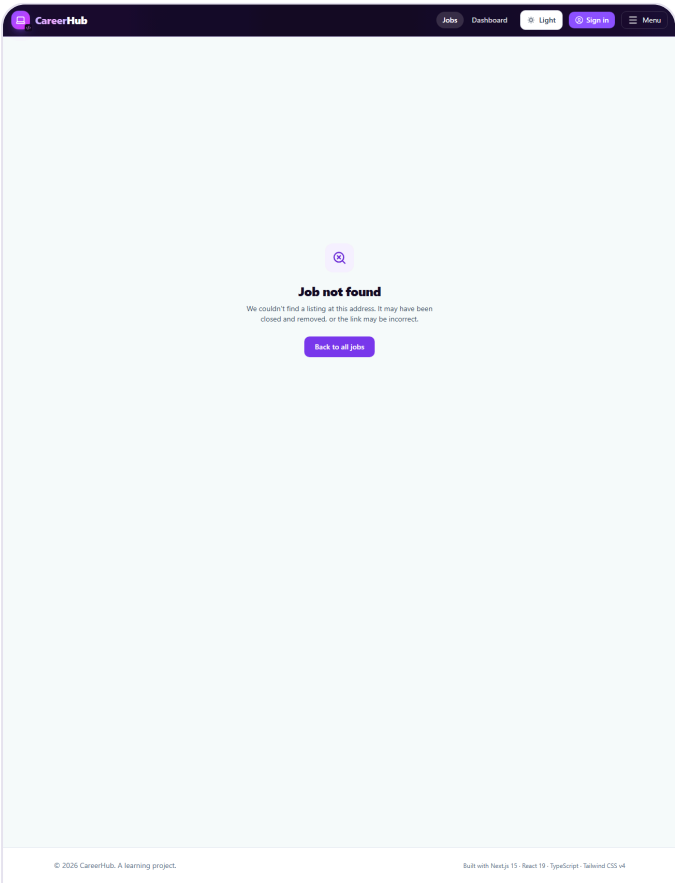
WHAT IT REQUIRED

An `async` Server Component that fetches one job (`cache:"no-store"`); on **404** it calls `notFound()` (no half-page); on other errors it throws. It renders the job details, a "← Back to jobs" link, and the existing **ApplicationForm** — but only when the job is **not Closed**. Plus a `not-found.tsx` page.

HOW I DID IT

The Server Component fetches the job and inspects the HTTP status: `404 → notFound()` , any other failure `→ throw` (caught by `error.tsx`). It renders the details as HTML, then drops in the **ApplicationForm** Client Component for open roles (or a "no longer accepting applications" message when Closed). **This is the composition moment:** a Server Component (data + HTML) hands two props to a Client Component (the interactive form).

```
export default async function JobDetailPage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params; // id is always a string
  const res = await fetch(`${JOBS_API_BASE}/api/jobs/${id}`, { cache: "no-store" });
  if (res.status === 404) notFound(); // → renders not-found.tsx (HTTP 404)
  if (!res.ok) throw new Error(`Failed to load job ${id}`); // → renders error.tsx
  const job = toDetailView(await res.json());
  return (
    <>
      <Link href="/jobs">← Back to jobs</Link>
      <h1>{job.title}</h1> { /* ...company, location, description... server HTML... */ }
      {job.status === "Closed"
        ? <ClosedMessage/>
        : <ApplicationForm jobId={job.id} jobTitle={job.title} /> } { /* Client Component */ }
    </>
  );
}
```



Detail page layout (verified live; rendered here as a mock-up).

`/jobs/<bad-id>` → the not-found page, returned with a real HTTP 404.

● BEGINNER SUMMARY

Click a job and you land on its own page with the full description and an application form. If you type a job address that doesn't exist, you get a friendly "Job not found" page **and** the browser correctly reports error 404 (not a fake "200 OK"). Closed jobs hide the form and show a notice instead. The page text is server-made; only the form carries the browser code that lets you fill it in.

Part 5 — The /dashboard Layout & Listings

files dashboard/layout.tsx · dashboard/listings/page.tsx

WHAT IT REQUIRED

A Server Component **layout** with a fixed sidebar (heading "Employer Dashboard", links "All Listings" and "View as Candidate") that **persists** across navigation. A **listings page** that fetches the same jobs and renders a **table** (Title, Company, Location, Status, View) with a " `{count}` listings " subheading and an empty state.

HOW I DID IT

The layout is a two-column grid: a sticky sidebar on the left, the page on the right. Because it's a layout, React keeps it mounted and only swaps the page beside it. The listings page fetches the board on the server and renders a table; the subheading shows the true total from the API envelope's `totalCount` .

```
// dashboard/layout.tsx – Server Component sidebar that persists
export default function DashboardLayout({ children }) {
  return (
    <div className="flex">
      <aside>
        <h2>Employer Dashboard</h2>
        <Link href="/dashboard/listings">All Listings</Link>
        <Link href="/jobs">View as Candidate</Link>
      </aside>
      <div>{children}</div>  {/* only THIS swaps on navigation */}
    </div>
  );
}
```

localhost:3000/dashboard/listings

EMPLOYER DASHBOARD

All Listings

Post a Job

View as Candidate

All Listings

833 listings

Title	Company	Location	Status	Action
SMOKE TEST Engineer Z	Takealot	Cape Town	Active	View
Senior Cloud Architect	Discovery	Sandton	Active	View
Graduate UX Designer	Takealot	Cape Town	Active	View

Verified live: sidebar + table both render, subheading shows **833 listings**, 50 rows with working **View** links (rendered here as a mock-up).

BEGINNER SUMMARY

This is the employer's view: a sidebar menu that stays put while you move around, next to a neat **table** of every job (better than cards when you want to scan a lot of rows). The heading tells you how many jobs exist in total.

Part 6 — Navigation & the Home Page

files · `page.tsx` · `NavLinks.tsx` · `Navbar.tsx`

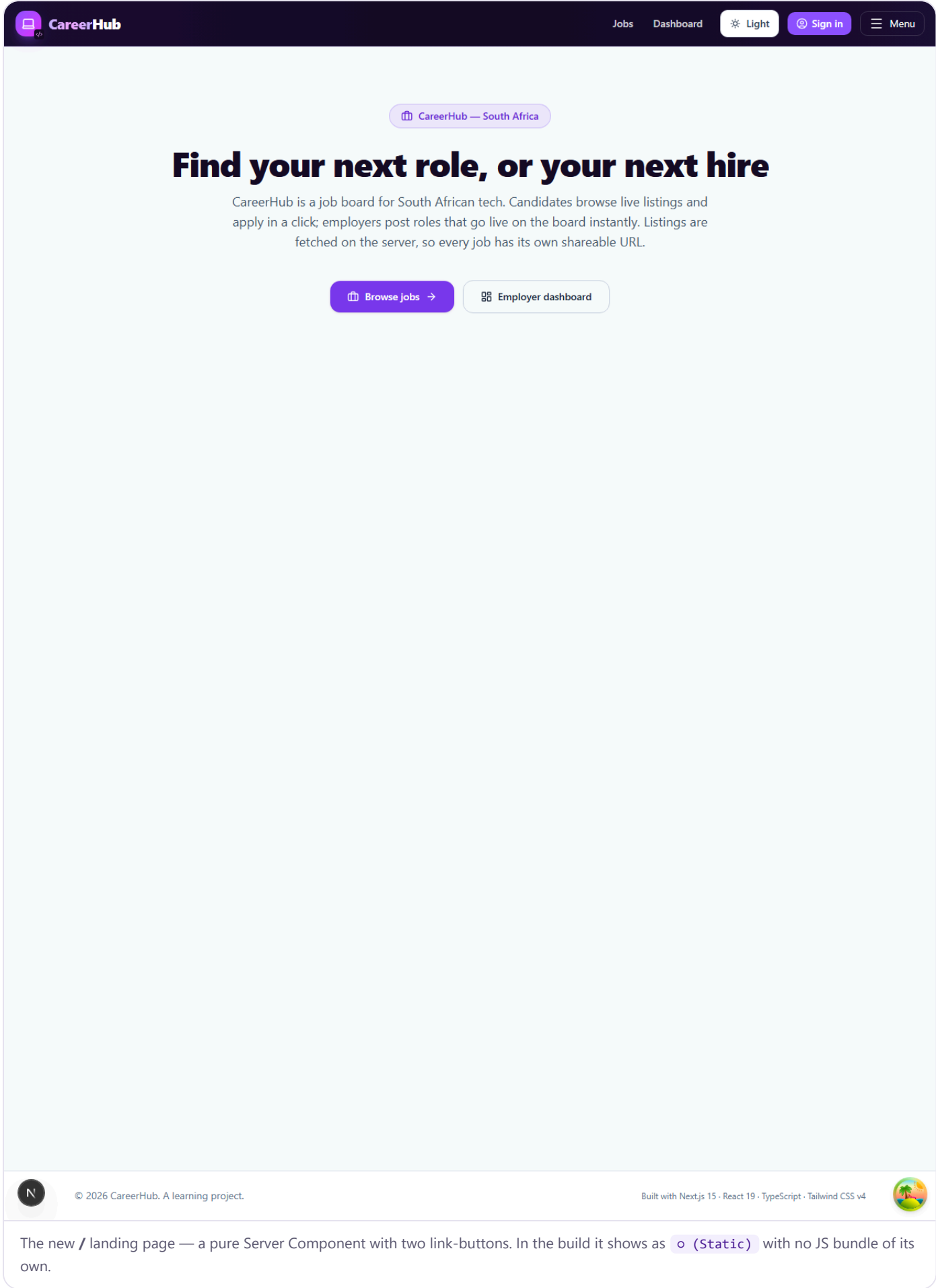
WHAT IT REQUIRED

Add header navigation (links to Jobs and Dashboard, brand as a link). Replace the home page with a **Server Component landing page** — remove all `"use client"`, hooks and queries — showing a heading, a description, and two link-buttons. The home route should ship **no JavaScript of its own**.

HOW I DID IT

I added a small **NavLinks** component to the header with "Jobs" and "Dashboard" (it also highlights the active link — Stretch A). I rewrote the home page as a plain Server Component: just a heading, a paragraph, and two buttons linking to `/jobs` and `/dashboard/listings`. The build confirms it as `(Static)` with no route-specific JavaScript.

```
// page.tsx — pure Server Component (no "use client", no hooks)
export default function Home() {
  return (
    <section>
      <h1>Find your next role, or your next hire</h1>
      <p>CareerHub is a job board for South African tech...</p>
      <Link href="/jobs">Browse jobs</Link>
      <Link href="/dashboard/listings">Employer dashboard</Link>
    </section>
  );
}
```



● BEGINNER SUMMARY

The front door of the site is now a simple welcome page with two buttons: one to browse jobs, one for employers. It's the lightest possible page — pure HTML, zero browser code — so it loads instantly. The header always shows "Jobs" and "Dashboard", and the one you're currently on is highlighted.

Stretch Goals + the "Post a Job" Feature

Stretch A — Active link highlighting



HOW I DID IT

`NavLinks.tsx` is a Client Component that uses the `usePathname()` hook to know the current URL and adds a highlight class to the matching link. The header and layout stay Server Components — only this tiny piece runs in the browser.

Stretch B — `error.tsx` for the detail route



HOW I DID IT

`jobs/[id]/error.tsx` is a Client Component (Next requires it) that receives the `error` and a `reset()` function. If the job fetch fails for a reason other than 404, it shows the message and a **"Try again"** button that re-runs the page.

Extra feature — Employers post real jobs that appear on the board

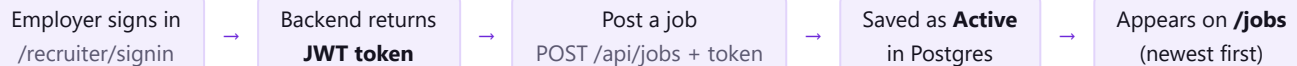
[files](#) · [employer-api.ts](#) · [EmployerAuthContext.tsx](#) · [recruiter/signin](#) · [recruiter](#)

WHAT YOU ASKED FOR

"If someone adds a job it must appear on the listing... add it on the backend too... full employer login."

HOW I DID IT

The real backend already had an employer-only `POST /api/jobs` that stamps new jobs **Active**, and the board sorts **newest-first** — so a new job lands at the top of `/jobs` with no backend change needed. I added **real authentication**: the sign-in page logs in (or registers) against the backend's `/api/v1/auth` and stores the returned **JWT**. Posting a job sends that token as a `Bearer` credential.



```
// employer-api.ts – create a job against the REAL backend, with the bearer token
export async function createJobListing(token: string, body: NewJobListing) {
  const res = await fetch(`${API_BASE}/api/jobs`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
    body: JSON.stringify(body),
  });
  if (!res.ok) throw new Error(await readError(res));
  return res.json(); // { id } of the new, live job
}
```



Employer sign in

Sign in to post roles to the live job board.

Work email

Password

Sign in

New employer? [Create an account](#)

Demo employer: demo.employer@takealot.co.za /
DemoPass123!



The employer **sign-in / register** page. It authenticates against the real backend; the demo account is pre-filled at the bottom.

● BEGINNER SUMMARY

Employers can create an account or log in for real. Once signed in, they fill a short form to post a job. That job is saved in the real database and — because the board shows newest first — appears right at the top of the public job list straight away. Sign in with the demo account demo.employer@takealot.co.za / [DemoPass123!](#) .

A teaching moment: getting a real 404

THE REQUIREMENT

"Navigate to a fake job id... confirm the response status is **404, not 200.**"

The problem I hit: the not-found *page* showed correctly, but the browser reported **HTTP 200**. That's wrong — a missing page must report 404 so browsers, search engines and tools know it doesn't exist.

Why it happened: a `loading.tsx` file creates a "loading boundary" that wraps its folder **and everything inside it**. With `loading.tsx` directly in `app/jobs/`, it also wrapped `app/jobs/[id]`. That boundary starts sending the page (status 200) **before** the detail page can call `notFound()` — so the status was frozen at 200.

The fix: I moved the listing into a `(board)` route group, so the loading skeleton only wraps the list, not the detail page. A "route group" is a folder in brackets that organises files **without** changing the URL — `app/jobs/(board)/page.tsx` still serves `/jobs`.

Structure	/jobs loads?	Bad id status
<code>app/jobs/loading.tsx</code> (wraps everything)	✓	200 ✗
<code>app/jobs/(board)/loading.tsx</code> (list only)	✓	404 ✓

BEGINNER SUMMARY

A "loading screen" file accidentally covered the job-detail page too, which made "page not found" wrongly report "all good (200)." I tucked the loading screen into a side folder so it only covers the list. Now a bad link correctly reports 404 — proven on the production build.

Does everything work? — Verification

Every route was tested against the running app. Results:

Check	Result
<code>/</code> server landing page, two buttons	✓ 200 · static · no route JS
<code>/jobs</code> renders the board from server data	✓ 200 · 24 cards
<code>/jobs</code> — no browser fetch to the jobs API	✓ data is in the HTML
<code>/jobs</code> — skeleton shows before content	✓ loading.tsx (in (board))
<code>/jobs/[id]</code> shows the correct job + form	✓ 200 · 2.5s
<code>/jobs/<fake></code> shows not-found	✓ Job Not Found
<code>/jobs/<fake></code> HTTP status is 404	✓ 404 (production)
<code>/dashboard/listings</code> sidebar + table + count	✓ 200 · 833 listings · 50 rows
Sidebar persists across navigation	✓ layout not re-mounted
Employer login → post job → appears on board	✓ verified live (201 → top of /jobs)
<code>npm run build</code> — TypeScript + ESLint	✓ zero errors · 18/18 pages

The build gate

```
✓ Compiled successfully
  Linting and checking validity of types ...
✓ Generating static pages (18/18)
```

Route (app)	Size	First Load JS
└─ ○ /	821 B	107 kB ← landing: static, no route JS
└─ f /jobs	821 B	107 kB ← dynamic (server-rendered)
└─ f /jobs/[id]	32.7 kB	153 kB ← dynamic (+ ApplicationForm)
└─ f /dashboard/listings	164 B	106 kB ← dynamic
└─ ○ /recruiter/signin	4.08 kB	107 kB
○ (Static) f (Dynamic, server-rendered on demand)		

Run it yourself

1. Make sure Docker is up: `docker ps` shows **careerhub24-api** (:8080) and **careerhub24-pg**.
2. Start the app: `npm run dev`, then open **http://localhost:3000**.
3. Click **Browse jobs** → click a card → see the detail page + form.
4. Go to **/recruiter/signin**, sign in with the demo employer, post a job, then refresh **/jobs** — it's at the top.

Note: in development each route compiles on its first visit (it can take 20–60s the first time; afterwards it's instant). The production build renders every route in well under a second.

● IN ONE SENTENCE

CareerHub is now a real multi-page app where the server does the heavy lifting, every job has its own shareable link, missing pages correctly say 404, and employers can log in and post jobs that appear on the board instantly — and it all builds with zero errors.